COMP6231

Orkut Sphere

Scalable Social Network Analysis

PRESENTED BY
DECODE

Table of Contents

WHY GRAPHS?

ORKUT+

NEBULA GRAPH

INTRODUCTION

SCALABILITY

DSD FEATURES

REPLICATION

FAULT TOLERANCE

DATA PARTITIONING

Abstract

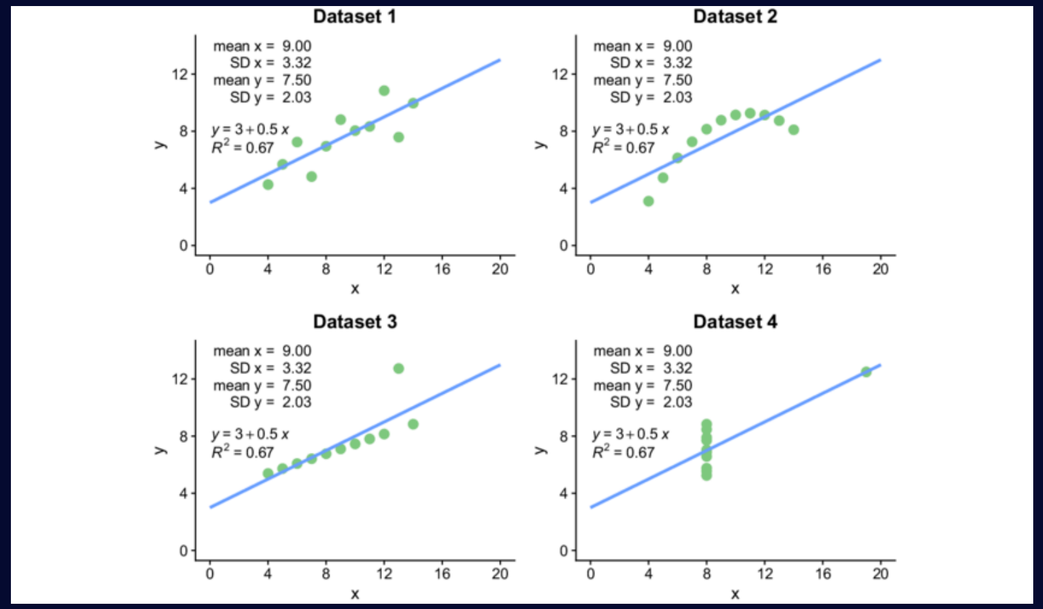
We built a distributed graph-analytics system using NebulaGraph to process the large Orkut dataset. With sharding, replication, and Raft consensus, the system scales horizontally and stays fault-tolerant. We implemented key analytics like community exploration, largest communities, shortest paths, and friend-based recommendations

- Distributed graph analytics using NebulaGrap
- Handles Orkut dataset (~3M users, 117M edges)
- Sharding + replication + Raft for scalability and reliability
- Analytics: communities, largest groups, shortest paths, recommendations

Why Graphs?

TL;DR

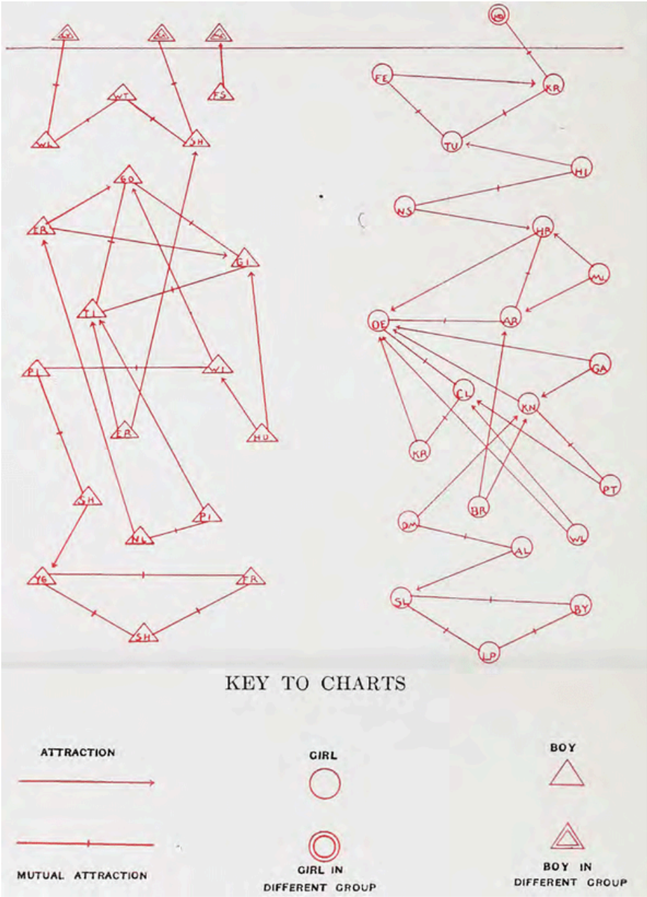
GRAPHS SHOW US RELATIONSHIPS THAT OTHER DATA STRUCTURES OR MEASUREMENTS MIGHT MISS



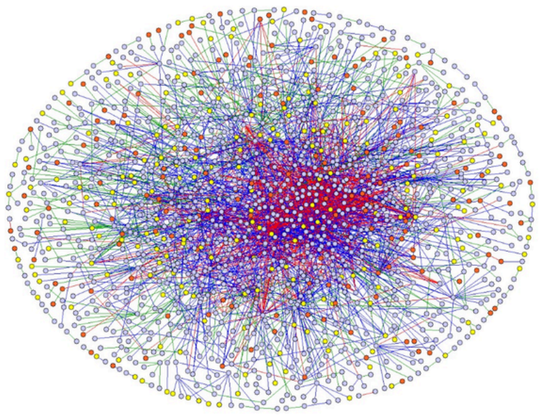
GRAPHS IN STATISTICAL ANALYSIS, ANSCOMBE 1973

DEMONSTRATED THAT MEANINGFUL VARIATIONS IN DATA CAN BE MISSED BY TRADITIONAL ANALYSIS TECHNIQUES

Graph Databases



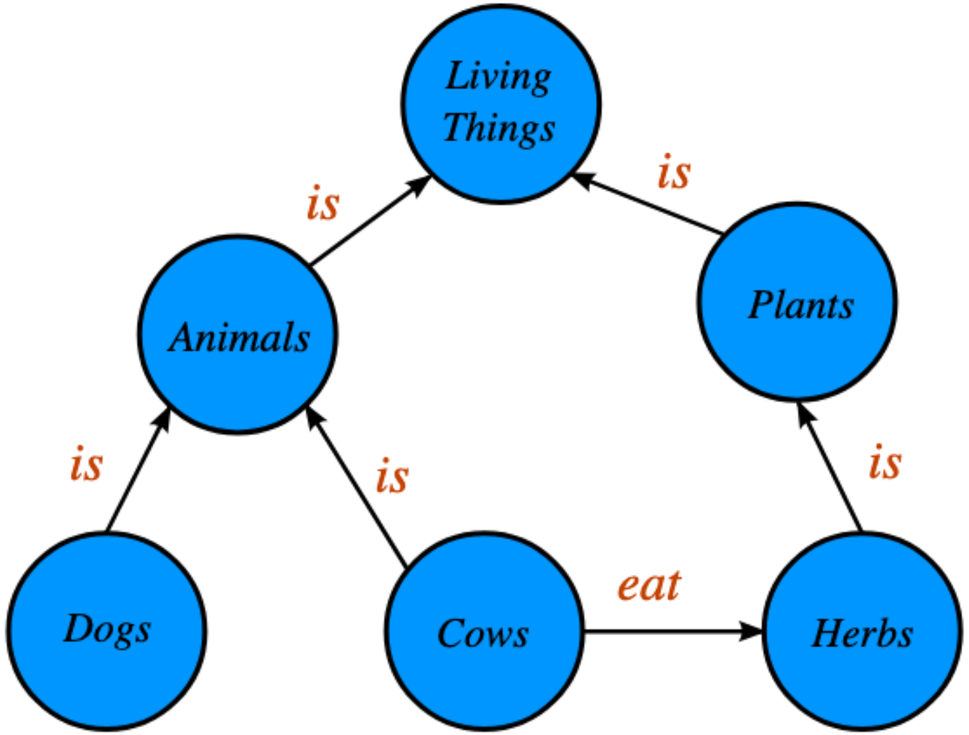
Moreno and Hale's Sociogram, 1934



We use graphs for applications that require deep dependencies, a context that is challenging for traditional databases.
A relevant example are knowledge graphs for LLMs.

To meet modern demands graph demands, we need distributed graph databases...

KNOWLEDGE GRAPHS



Dataset Overview

Dataset 1: Friendship Graph

- 3 Million Users
- 117 Million Friend Edges
- Undirected graph
- File: com-orkut.ungraph.txt

Dataset 2: Top 5000 Communities

- 5000 real social groups
- Each line = list of user IDs in that community
- File: com-orkut.top5000.cmty.txt

Dataset statistics	
Nodes	3072441
Edges	117185083
Nodes in largest WCC	3072441 (1.000)
Edges in largest WCC	117185083 (1.000)
Nodes in largest SCC	3072441 (1.000)
Edges in largest SCC	117185083 (1.000)
Average clustering coefficient	0.1666
Number of triangles	627584181
Fraction of closed triangles	0.01414
Diameter (longest shortest path)	9
90-percentile effective diameter	4.8

Understanding User & Community Behavior Through Graph Queries

Modern social platforms (Facebook, LinkedIn, Instagram, Reddit) rely on graph queries to understand friendships, communities, and influence.
Our Orkut graph and the following queries demonstrate the same real-world capabilities.

Visualizing a User's Social Circle

- Shows a user's immediate (1-hop) network and their connections.
- Helps platforms display friend graphs, trust networks, and profile relationships.

Multi-Hop Network Expansion

- Reveals 1-hop, 2-hop, and 3-hop reachability from a user.
- Used to model influence, spread, trust, and network depth.

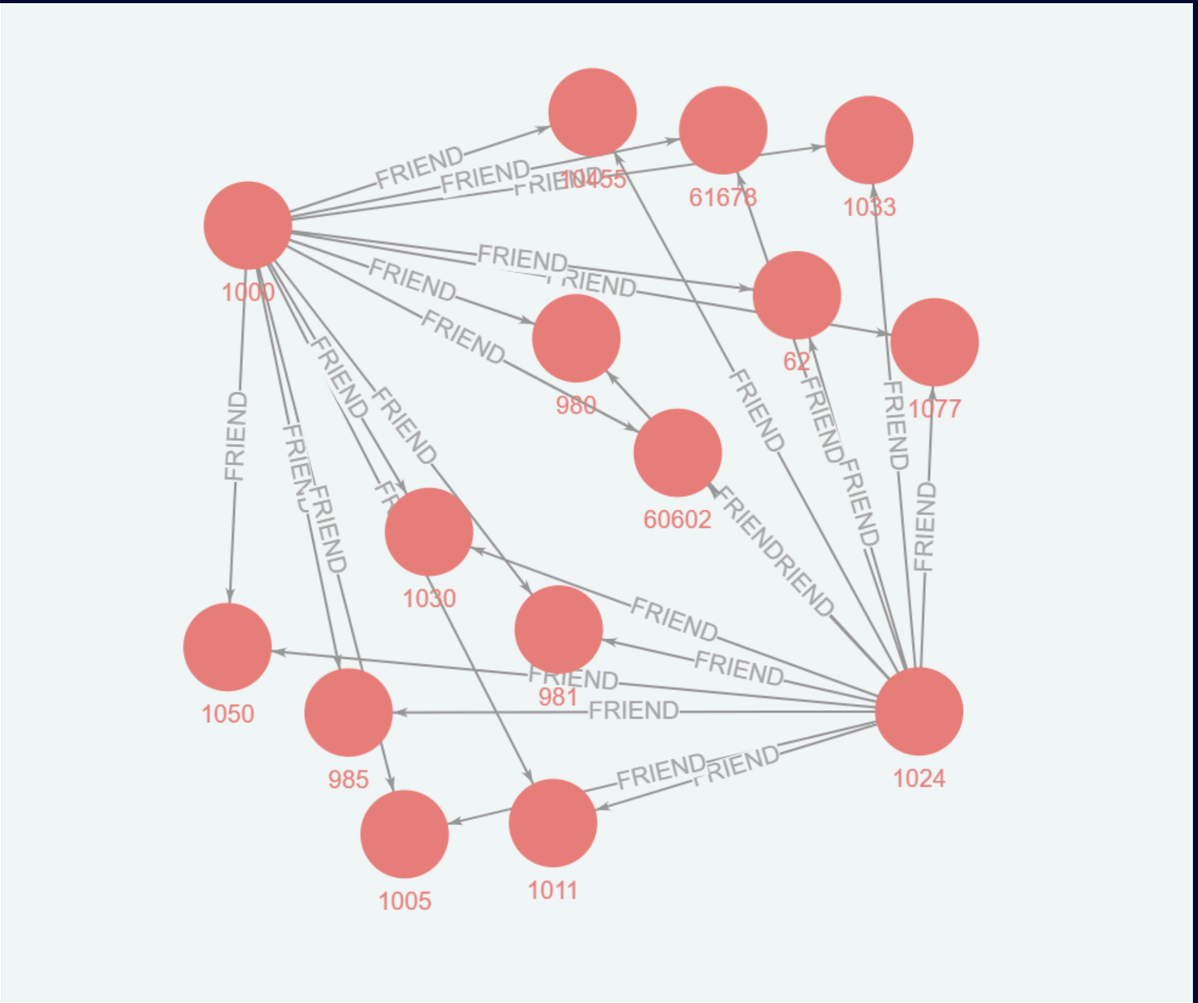
Finding Mutual Friends Between Users

- Core of "People You May Know" - identifies overlapping social circles between two users.
- Forms the basis of connection-recommendation logic in real platforms.

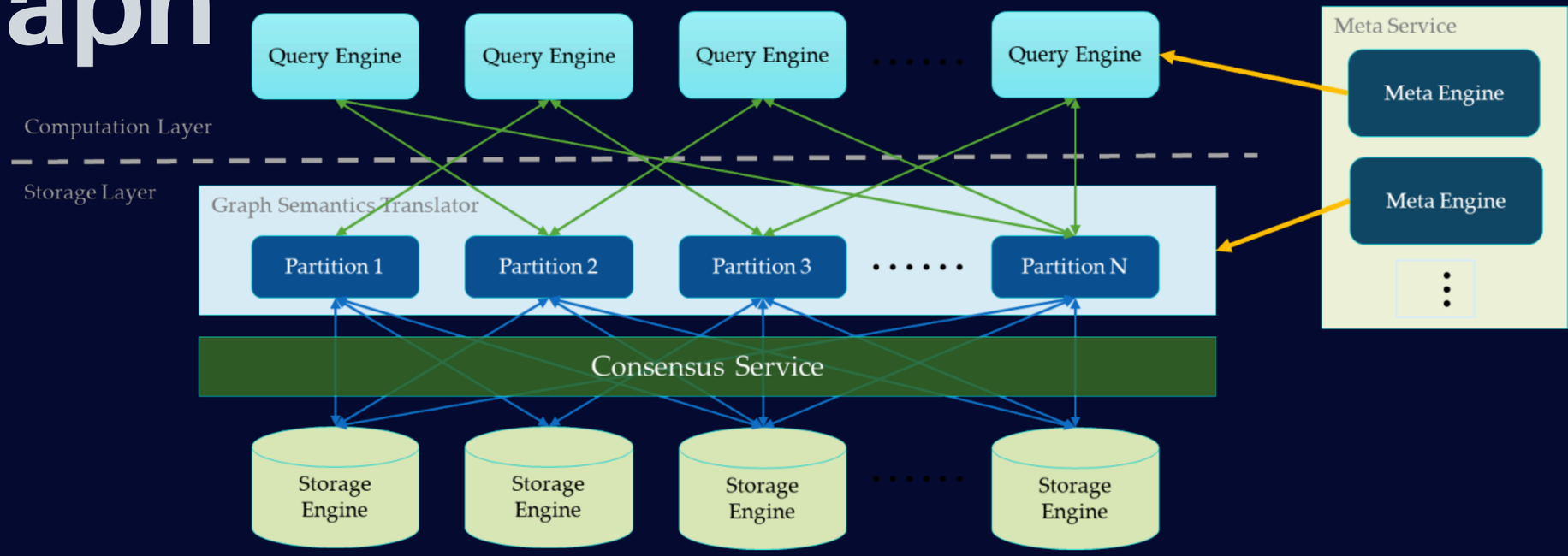
Community Recommendation Logic

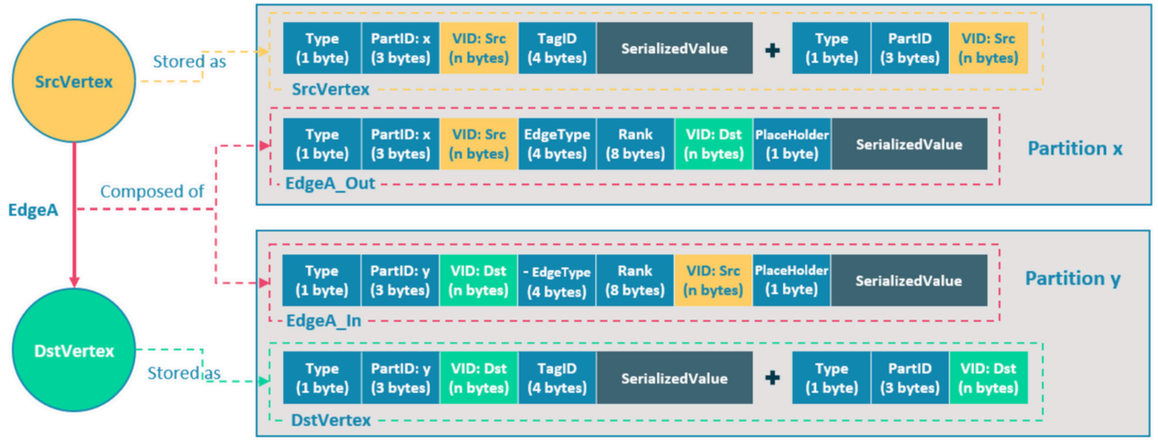
- Suggests groups that a user is likely to join based on their friends' memberships.
- Mirrors real recommendation engines in social platforms.

Mutual Friend Discovery



NebulaGraph





Why NebulaGraph?

Feature	NebulaGraph	Neo4j
Distributed Architecture	Yes (native sharding & replication)	No (only with paid Enterprise clustering)
Horizontal scaling	Scales by adding nodes	Limited — mainly single-node focused
Consensus	Built-in Raft for leader election	Requires separate clustering components
Optimized for large graphs	Designed for billions of edges	Struggles at massive scale

DSD

NebulaGraph's architecture is robust for distributed applications, here are the features we studied and tested.

- Scaling
- Data partitioning
- Replication
- Fault tolerance

Data Partitioning

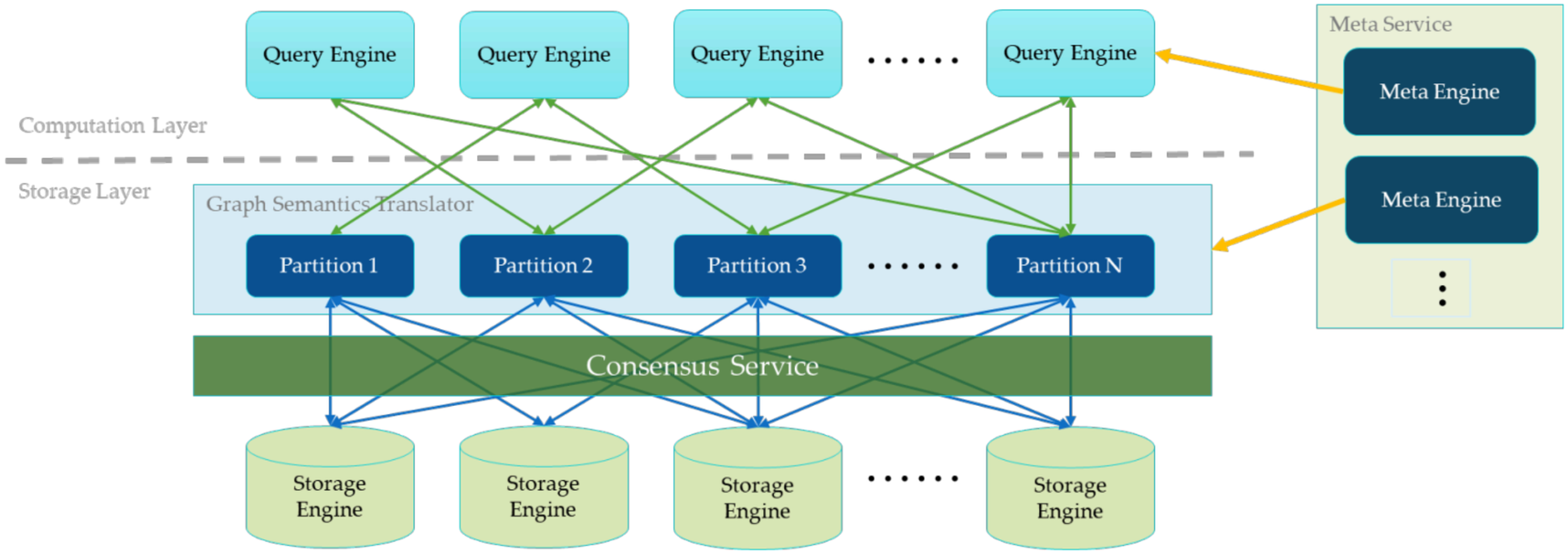
```
[orkut]> SHOW HOSTS;
```

Table

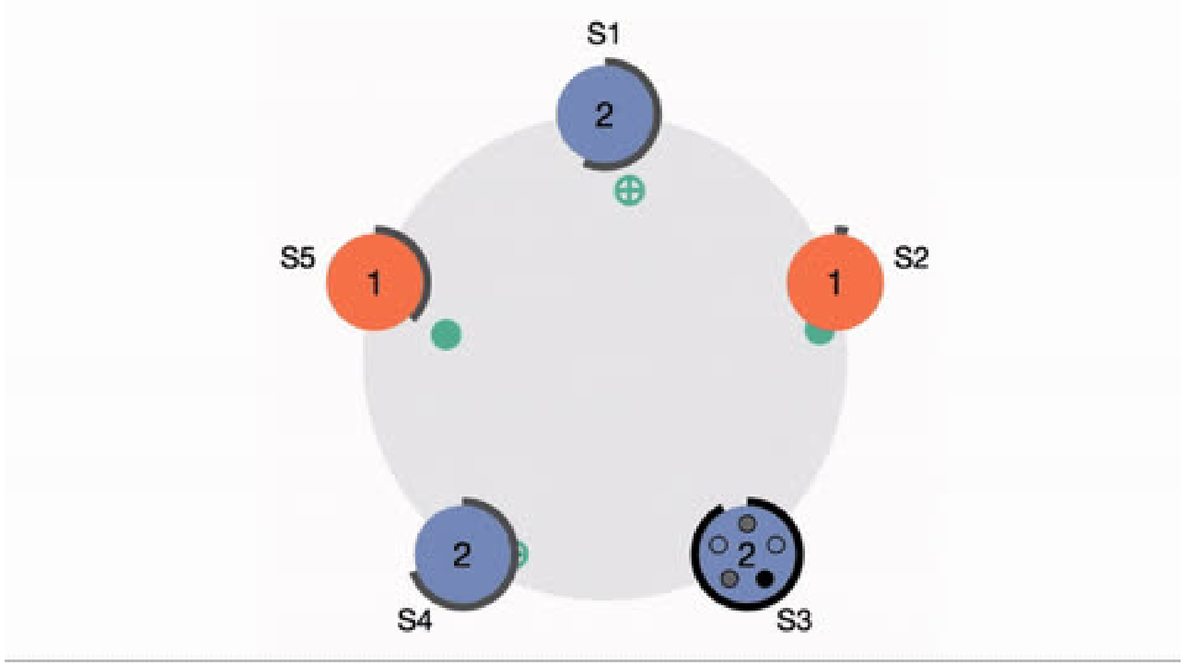
Host	Port	Status	Leader count	Leader distribution	Partition distribution	Version
storaged0	9779	ONLINE	19	orkut:19	orkut:60	3.6.0
storaged1	9779	ONLINE	22	orkut:22	orkut:60	3.6.0
storaged2	9779	ONLINE	24	orkut:24	orkut:60	3.6.0
storaged3	9779	ONLINE	20	orkut:20	orkut:60	3.6.0
storaged4	9779	ONLINE	15	orkut:15	orkut:60	3.6.0

Total 5 1

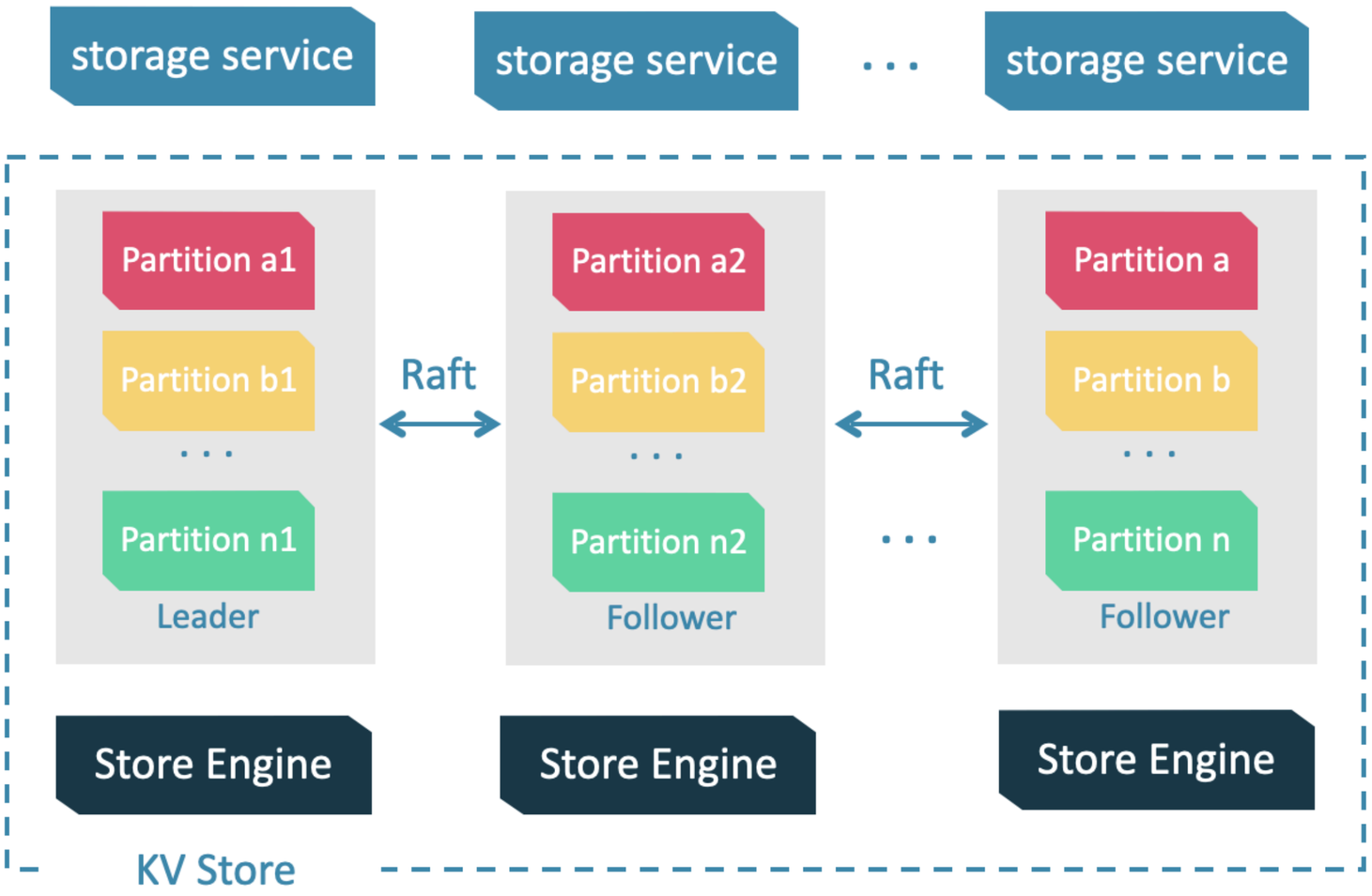
Replication



Fault Tolerance: RAFT



Scaling


storage service
storage service
. . .
storage service
Partition a1
Partition b1
. . .
Partition n1
Leader
Raft
Partition a2
Partition b2
. . .
Partition n2
Follower
Raft
. . .
Partition a
Partition b
. . .
Partition n
Follower
Store Engine
Store Engine
Store Engine
KV Store

DEMO TIME